Contents

16. DBMS & SQL Complete Fundamentals (21,Oct,2026) 2
16.1. DBMS & SQL Topic Index 38
16.2. Keys and Numeric Functions 42
16.3. Keys and Numerical Functions(detailed) 43
16.4. Date Functions Summary 56
16.5. Date Functions — Full Detailed Notes 59
16.6. Window Functions — Summary 71
16.7. Window Functions — Full Detailed Notes 72

16. DBMS & SQL Complete Fundamentals (21,Oct,2026)

Beginner-friendly notes • Definitions • Syntax • Examples • Differences

These notes follow your exact 14 groups and original 38 numbered topics. Each topic is explained in simple English with examples, without adding unrelated subject groups.

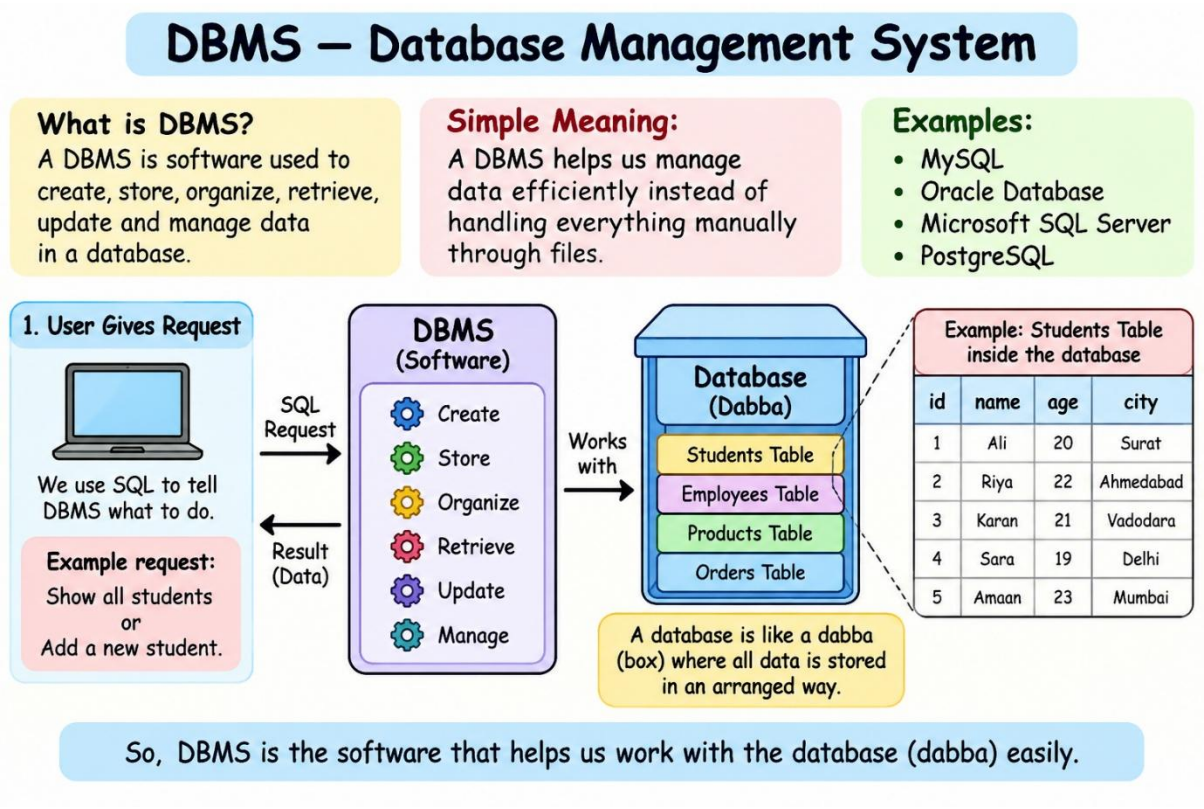
Group 1 — Database Fundamentals

1. DBMS — Database Management System

Definition: A DBMS is software used to create, store, organize, retrieve, update and manage data in a database.

Simple meaning: A DBMS helps us manage data efficiently instead of handling everything manually through files.

Examples: MySQL, Oracle Database, Microsoft SQL Server and PostgreSQL.

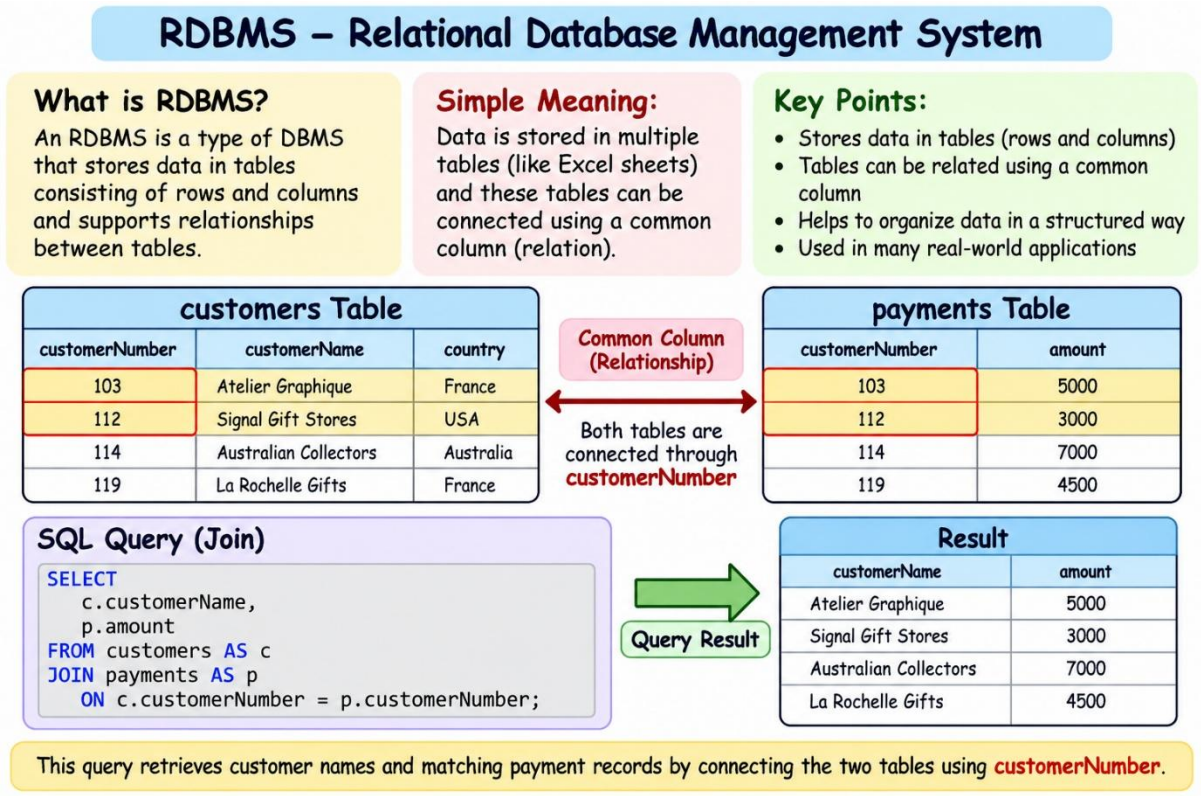


Use cases:

- Banking: accounts and transactions.
- E-commerce: customers, products, orders and payments.
- Hospitals: patients and appointments.

- Education: student details, marks and attendance.
- Inventory: products and stock quantities.

2. RDBMS — Relational Database Management System



Definition: An RDBMS is a type of DBMS that stores data in tables consisting of rows and columns and supports relationships between tables.

Example:

customers

customerNumber	customerName	country
103	Atelier Graphique	France
112	Signal Gift Stores	USA

payments

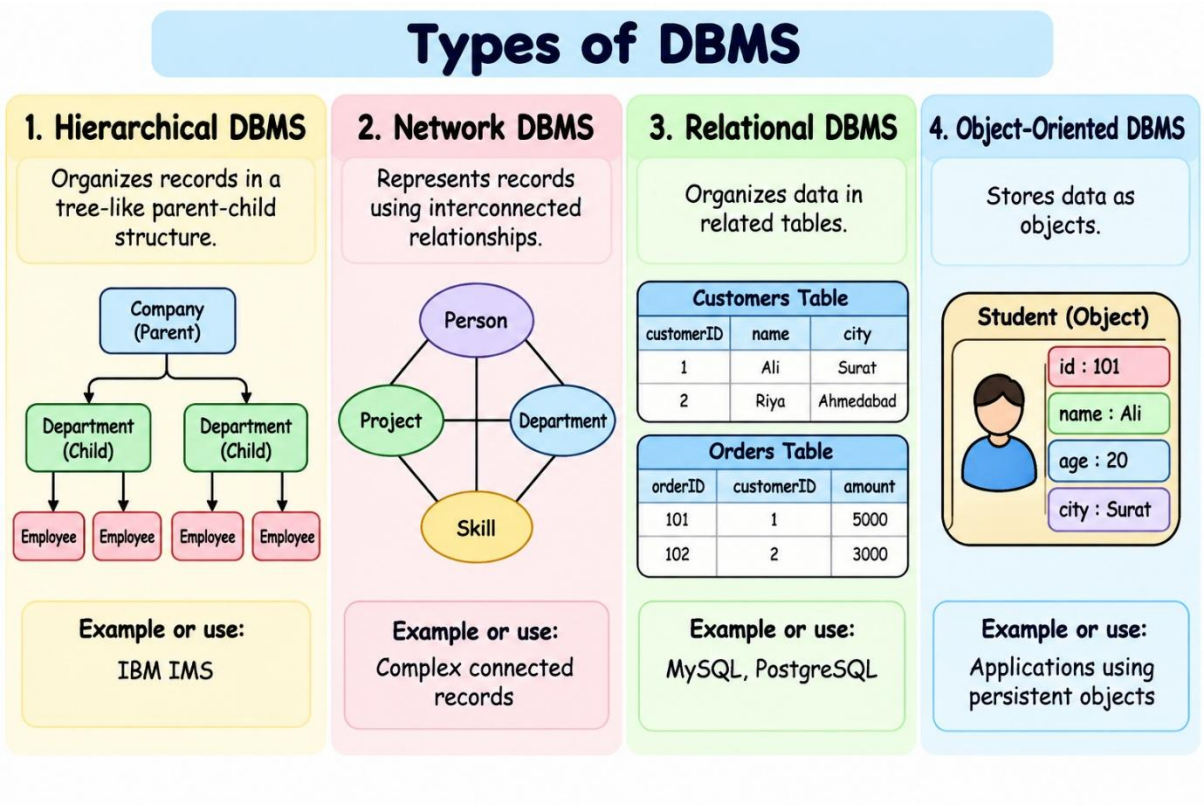
customerNumber	amount
103	5000
112	3000

The tables can be connected through customerNumber.

```
SELECT
   c.customerName,
   p.amount
FROM customers AS c
JOIN payments AS p
   ON c.customerNumber = p.customerNumber;
```

This query retrieves customer names and matching payment records.

3. Types of DBMS



Types of DBMS

1. What are the Types of DBMS?

A DBMS (Database Management System) is software used to store, organize, retrieve, update and manage data.

Different types of DBMS organize data in different ways.

The figure shows four types:

1. Hierarchical DBMS
2. Network DBMS

3. Relational DBMS
4. Object-Oriented DBMS

2. Hierarchical DBMS — Tree Structure

Definition: A Hierarchical DBMS organizes records in a tree-like parent-child structure.

Imagine a company:

Company (Parent)

Department

Department

Employee

Employee

How it works:

- A company is the parent.
- Departments are children of the company.
- Employees belong under departments in this example.

The data follows a tree-like structure.

Example: IBM IMS.

Remember: Hierarchical DBMS = Parent and child tree.

3. Network DBMS — Connected Structure

Definition: A Network DBMS represents records using interconnected relationships.

In the figure, we see four examples: Person, Project, Department and Skill.

Person

Project

Department

Skill

The lines represent relationships between records or entities.

What are Person, Project, Department and Skill?

These are examples of entities in the figure.

An entity represents a person, thing, place or concept about which we want to store data.

Entity	**Example of actual data**	**Possible attributes (columns)**
Person	Ali	person_id, person_name
Project	Website Project	project_id, project_name
Department	IT	department_id, department_name
Skill	SQL	skill_id, skill_name

Understand these three terms:

- Entity: The type of thing we want to store data about, such as Person.
- Record: One actual entry, such as the person Ali.
- Attribute (column): A property of that record, such as person_name.

So, Person, Project, Department and Skill are not databases or columns. They represent different entity types in the diagram.

What is the purpose of an entity?

An entity helps us identify what kind of information we want to store and how that information can relate to other entities.

For example, a person may work on a project and have a particular skill. These are connections between different entities.

A Network DBMS allows records to have multiple connections instead of restricting every record to a simple tree structure.

Example or use: Managing complex, interconnected records.

Remember: Network DBMS = Connected records.

4. Relational DBMS — Related Tables

Definition: A Relational DBMS organizes data in tables made of rows and columns. Tables can be connected through a common column.

For example, the figure shows a Customers table and an Orders table.

Customers table

customerID	name	city
1	Ali	Surat
2	Diya	Ahmedabad

Orders table

orderID	customerID	amount
101	1	5000
102	2	3000

Notice that both tables contain customerID.

This common column connects the tables:

- Customer 1 is Ali, and the matching order amount is 5000.
- Customer 2 is Diya, and the matching order amount is 3000.

What is the benefit of this relationship?

Instead of storing all customer information repeatedly in every order, we can keep related information in separate tables and connect it when needed.

Examples: MySQL and PostgreSQL.

Remember: Relational DBMS = Related tables connected through common columns.

5. Object-Oriented DBMS — Objects

Definition: An Object-Oriented DBMS stores data as objects.

In the figure, a student is shown as one object containing different pieces of information.

Student Object

ID: 101

Name: Ali

Age: 20

City: Surat

How it works:

The student is represented as one object, and that object contains information such as ID, name, age and city.

- The object represents the student.
- The information inside it describes the student.
- The stored object can be retrieved later.

Example or use: Applications that work with persistent objects, meaning objects whose data is saved and remains available later.

Remember: Object-Oriented DBMS = Data stored as objects.

6. Final Revision Table

Type	**How data is organized**	**Example or use**
Hierarchical DBMS	Tree-like parent-child structure	IBM IMS
Network DBMS	Interconnected records and relationships	Complex connected records
Relational DBMS	Related tables of rows and columns	MySQL, PostgreSQL
Object-Oriented DBMS	Objects containing data	Applications using persistent objects

One-line summary

- Hierarchical: Data is organized like a tree.
- Network: Records are connected in multiple ways.
- Relational: Data is stored in related tables.
- Object-Oriented: Data is stored as objects.

Most important concept: An entity represents what we want to store data about; an attribute describes it; a record contains one actual entry; and a relationship connects entities or records.

Type	**Definition**	**Example or use**
Hierarchical DBMS	Organizes records in a tree-like parent-child structure	IBM IMS

Network DBMS	Represents records using interconnected relationships	Complex connected records
Relational DBMS	Organizes data in related tables	MySQL, PostgreSQL
Object-Oriented DBMS	Stores data as objects	Applications using persistent objects
Document Database	Stores records as documents, often JSON-like	MongoDB
Key-Value Database	Stores values associated with unique keys	Redis
Column-Family Database	Organizes data into column families	Cassandra
Graph Database	Represents entities as nodes and relationships as edges	Neo4j

The first three are commonly taught as traditional DBMS categories. Document, key-value, column-family and graph databases are commonly associated with NoSQL database models.

4. Difference Between DBMS and RDBMS

DBMS	**RDBMS**
General term for database management software	A type of DBMS based on the relational model
Data organization depends on the database model	Data is organized into related tables
Relationships depend on the system	Relationships are commonly represented through keys
Constraints and transaction support vary by system	Relational constraints and transactions are commonly supported
Can include relational or non-relational systems	Examples include MySQL and PostgreSQL

Remember: Every RDBMS is a DBMS, but not every DBMS is an RDBMS.

5. Use Cases of DBMS

Industry	What a DBMS manages
Banking	Accounts, balances and transactions
E-commerce	Customers, products and orders
Healthcare	Patient records and appointments
Education	Student details, marks and attendance
Inventory	Product details and stock
Airlines	Flights, bookings and passengers
Business	Sales, payments and customer records

Main purpose: To store and retrieve data reliably and keep it organized.

Group 2 — SQL Fundamentals

6. SQL — Structured Query Language

Definition: SQL is a language used to define, retrieve and manipulate data in relational databases.

Example:

SELECT *

FROM customers;

This retrieves all columns and rows from the customers table.

SQL can be used for tasks such as:

- Creating databases and tables.
- Inserting new records.
- Updating existing records.
- Deleting records.
- Retrieving and analyzing data.

7. Types of SQL Commands

SQL commands are commonly divided into five categories.

Category	Full form	Purpose	Examples

DDL	Data Definition Language	Defines or changes database structures	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	Adds, modifies or deletes data	INSERT, UPDATE, DELETE
DQL	Data Query Language	Retrieves data	SELECT
DCL	Data Control Language	Controls permissions	GRANT, REVOKE
TCL	Transaction Control Language	Controls transactions	COMMIT, ROLLBACK

The distinction between DQL and DML can vary by teaching convention; some references classify SELECT as DML.

8. CRUD Operations

Definition: CRUD represents the four basic operations used to manage data.

Letter	**Operation**	**Purpose**	**SQL command**
C	Create	Add a new record	INSERT
R	Read	Retrieve records	SELECT
U	Update	Modify records	UPDATE
D	Delete	Remove records	DELETE

Example:

```
-- Create
INSERT INTO customers
  (customerNumber, customerName, country)
VALUES
  (501, 'Sample Shop', 'USA');


-- Read
SELECT *
```

```
FROM customers;


-- Update
UPDATE customers
SET country = 'France'
WHERE customerNumber = 501;


-- Delete
DELETE FROM customers
WHERE customerNumber = 501;
```

Important difference: CRUD's Create operation normally means inserting a record. SQL's CREATE TABLE command creates a table structure instead.

Group 3 — SQL Commands

9. DDL — Data Definition Language

DDL commands create or change database structures.

A. CREATE

Definition: Creates a database object, such as a database, table or view.

```
CREATE DATABASE shop_db;
```

Creates a database named shop_db.

```
CREATE TABLE customers (
  customerNumber INT PRIMARY KEY,
  customerName VARCHAR(50),
  country VARCHAR(30)
);
```

Creates a table with three columns.

B. DROP

Definition: Removes a database object and its structure.

```
DROP TABLE customers;
```

Removes the customers table and its data.

C. ALTER

Definition: Changes the structure of an existing database object.

Add a column:

```
ALTER TABLE customers
ADD COLUMN city VARCHAR(50);
```

Modify a column's data type:

```
ALTER TABLE customers
MODIFY COLUMN city VARCHAR(100);
```

D. TRUNCATE

Definition: Removes all rows from a table while retaining the table structure.

```
TRUNCATE TABLE customers;
```

The table remains, but its rows are removed.

Difference between DDL commands

Command	What it does
CREATE	Creates a database object
ALTER	Changes an object's structure
TRUNCATE	Removes all rows but retains the table
DROP	Removes the object itself

Safety: TRUNCATE and DROP are destructive operations. In MySQL, they also cause implicit transaction commits.

10. DML — Data Manipulation Language

DML commands manipulate records stored in tables.

A. INSERT

Definition: Adds new rows.

```
INSERT INTO customers
  (customerNumber, customerName, country)
VALUES
```

(502, 'Example Store', 'USA');

B. UPDATE

Definition: Modifies existing rows.

UPDATE customers

SET country = 'France'

WHERE customerNumber = 502;

This updates the matching customer.

Remember: Without an appropriate WHERE condition, an UPDATE can change every row.

C. DELETE

Definition: Removes rows.

DELETE FROM customers

WHERE customerNumber = 502;

This removes the matching customer record, subject to any applicable foreign-key restrictions.

Without WHERE, the statement attempts to delete all rows.

11. DQL — Data Query Language

Definition: DQL is the category commonly used to describe commands that retrieve data from a database.

Its main command is SELECT.

SELECT customerName, country

FROM customers;

This displays customer names and countries without changing the stored data.

Another example:

SELECT *

FROM customers

WHERE country = 'USA';

This retrieves only USA customers.

Group 4 — Data Types

12. SQL Data Types

Definition: A data type specifies what kind of value a column can store.

A. Numeric Data Types

Data type	Purpose	Example
INT	Whole numbers	25
BIGINT	Large whole numbers	Large identifiers
SMALLINT	Smaller-range whole numbers	Small counts
TINYINT	Small-range integers	Small status values
DECIMAL(p,s)	Exact decimal values	1500.75
FLOAT	Approximate floating-point values	Measurements
DOUBLE	Approximate floating-point values	Scientific calculations

Example:

creditLimit DECIMAL(10,2),

quantity INT

Use DECIMAL for money when exact decimal arithmetic is required.

B. String Data Types

Data type	Purpose
CHAR(n)	Fixed-length text
VARCHAR(n)	Variable-length text
TEXT	Longer text

Example:

customerName VARCHAR(50),

country VARCHAR(30)

VARCHAR(50) can store a string of up to 50 characters.

C. Date and Time Data Types

Data type	Purpose
DATE	Date only

TIME	Time only
DATETIME	Date and time
TIMESTAMP	Date-time value with MySQL-specific timestamp behavior
YEAR	Year value

Example:

orderDate DATE

Easy rule: Choose a data type that matches the kind of data you intend to store.

Group 5 — SELECT Query Clauses

13. SELECT

Definition: Specifies the columns or expressions to return.

```
SELECT customerName, country
FROM customers;
```

Use SELECT * when you want all columns:

```
SELECT *
FROM customers;
```

14. FROM

Definition: Specifies the source table or other data source.

```
SELECT *
FROM customers;
```

Here, customers is the source table.

15. WHERE

Definition: Filters individual rows based on a condition.

```
SELECT *
FROM customers
WHERE country = 'USA';
```

Only customers whose country is USA are returned.

Other examples:

```
WHERE creditLimit > 50000
```

```
WHERE country = 'USA'
  AND creditLimit > 50000
```

Key idea: WHERE answers the question, “Which rows do I want?”

16. JOIN

Definition: Combines related rows from two or more tables.

INNER JOIN

Returns only rows that match the join condition.

```
SELECT
   c.customerName,
   p.amount
FROM customers AS c
JOIN payments AS p
   ON c.customerNumber = p.customerNumber;
```

LEFT JOIN

Returns all rows from the left table and matching rows from the right table. Missing matches produce NULL values for right-table columns.

```
SELECT
   c.customerName,
   p.amount
FROM customers AS c
LEFT JOIN payments AS p
   ON c.customerNumber = p.customerNumber;
```

RIGHT JOIN

Returns all rows from the right table and matching rows from the left table.

CROSS JOIN

Returns combinations of rows from both tables.

SELF JOIN

Joins the same table to itself using different aliases.

```
SELECT
  e.employeeNumber,
  CONCAT(e.firstName, ' ', e.lastName) AS EmployeeName,
  CONCAT(m.firstName, ' ', m.lastName) AS ManagerName
FROM employees AS e
LEFT JOIN employees AS m
  ON e.reportsTo = m.employeeNumber;
```

Here, e represents an employee and m represents their manager.

Remember: In the Classic Models schema, an employee's reportsTo column matches the manager's employeeNumber.

17. DISTINCT

Definition: Removes duplicate rows from the selected result.

```
SELECT DISTINCT country
FROM customers;
```

Displays each country once.

By comparison:

```
SELECT country
FROM customers;
```

May display the same country repeatedly because many customers can belong to one country.

18. GROUP BY

Definition: Groups rows with matching values so aggregate calculations can be performed separately for each group.

```
SELECT
  country,
  COUNT(*) AS TotalCustomers
FROM customers
GROUP BY country;
```

This returns one customer count for each country.

Another example:

```
SELECT
  customerNumber,
  SUM(amount) AS TotalPayments
FROM payments
GROUP BY customerNumber;
```

This calculates the total payments separately for each customer.

WISE rule: Country-wise count usually needs GROUP BY country. But a request for the actual customers of one country generally needs WHERE, not GROUP BY.

19. HAVING

Definition: Filters grouped results using conditions, often involving aggregate calculations.

```
SELECT
  customerNumber,
  SUM(amount) AS TotalPayments
FROM payments
GROUP BY customerNumber
HAVING SUM(amount) > 10000;
```

This returns customers whose total payments exceed 10,000.

WHERE vs. HAVING

WHERE	**HAVING**
Filters individual rows	Filters groups
Used before grouping	Used after grouping
Example: WHERE country = 'USA'	Example: HAVING SUM(amount) > 10000

20. ORDER BY

Definition: Sorts the result.

Highest to lowest:

```
ORDER BY creditLimit DESC;
```

Lowest to highest:

ORDER BY creditLimit ASC;

- DESC means descending.
- ASC means ascending and is the default direction.

21. LIMIT

Definition: Restricts the number of rows returned.

```
SELECT *
FROM customers
LIMIT 5;
```

Returns up to five rows.

For the top five customers by credit limit:

```
SELECT *
FROM customers
ORDER BY creditLimit DESC
LIMIT 5;
```

Important: LIMIT 5 alone does not identify the top five by a particular measure. Use ORDER BY to define the ranking.

Common SELECT Query Structure

```
SELECT columns
FROM table_name
JOIN another_table
  ON join_condition
WHERE row_condition
GROUP BY grouping_columns
HAVING group_condition
ORDER BY sorting_columns
LIMIT number_of_rows;
```

Use only the clauses required by the question.

Group 6 — Constraints

22. Constraints

Definition: Constraints are rules applied to columns or tables to help maintain valid and consistent data.

A. UNIQUE

Requires values to be unique within a constrained column or column combination.

email VARCHAR(100) UNIQUE

In MySQL, a UNIQUE column can generally contain multiple NULL values.

B. NOT NULL

Prevents a column from storing NULL.

customerName VARCHAR(50) NOT NULL

A customer name must be supplied.

C. PRIMARY KEY

Uniquely identifies every row in a table.

customerNumber INT PRIMARY KEY

A primary key cannot contain duplicate values or NULL.

D. DEFAULT

Provides a default value when an insert omits the column or explicitly requests DEFAULT.

status VARCHAR(20) DEFAULT 'Active'

If no status is supplied, the default is Active.

Complete example

```
CREATE TABLE example_customers (
  customerNumber INT PRIMARY KEY,
  customerName VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE,
  status VARCHAR(20) DEFAULT 'Active'
);
```

This table uses four constraints.

Group 7 — Keys

23. Primary Key

Definition: A primary key is a column, or combination of columns, that uniquely identifies each row in a table.

Example:

```
CREATE TABLE students (
  studentID INT PRIMARY KEY,
  studentName VARCHAR(50)
);
```

Here, studentID uniquely identifies each student.

Rules of a Primary Key

1. It must be unique.
2. It cannot contain NULL.
3. A table has one primary key definition, though it can contain multiple columns.
4. It can be referenced by foreign keys in other tables.

Why is a Primary Key important?

Without a reliable unique identifier, it can be difficult to identify and update the correct record when several records have similar names or details.

Group 8 — SQL Functions

24. Aggregate Functions

Definition: Aggregate functions calculate a summary from multiple rows. With GROUP BY, they produce a separate calculation for each group.

Function	Purpose
COUNT()	Counts rows or non-NULL values
SUM()	Calculates a total
AVG()	Calculates an average
MIN()	Returns the minimum value
MAX()	Returns the maximum value

Examples

```
SELECT COUNT(*) AS TotalCustomers
FROM customers;
```

Counts all customer rows.

```
SELECT SUM(amount) AS TotalPayments
FROM payments;
```

Adds payment amounts.

```
SELECT AVG(amount) AS AveragePayment
FROM payments;
```

Calculates the average payment amount.

```
SELECT MIN(amount) AS SmallestPayment,
    MAX(amount) AS LargestPayment
FROM payments;
```

Returns the smallest and largest payment values.

Important: COUNT(*) counts rows, including rows containing NULL in some columns. COUNT(column) counts non-NULL values in that column.

25. Date Functions

Date functions work with dates, times or date-time values.

Function	**Purpose**	**Example**
CURDATE()	Current date	SELECT CURDATE();
NOW()	Current date and time	SELECT NOW();
YEAR()	Extracts the year	YEAR(orderDate)
MONTH()	Extracts the month number	MONTH(orderDate)
DAY()	Extracts the day of the month	DAY(orderDate)
DATE_ADD()	Adds an interval	DATE_ADD(CURDATE(), INTERVAL 7 DAY)
DATE_SUB()	Subtracts an interval	DATE_SUB(CURDATE(), INTERVAL 2 MONTH)

DATEDIFF()	Difference between dates in days	DATEDIFF(date1, date2)
DATE_FORMAT()	Formats a date as text	DATE_FORMAT(orderDate, '%Y-%m')

Example — Orders from the Last Two Months

```
SELECT *
FROM orders
WHERE orderDate >= DATE_SUB(CURDATE(), INTERVAL 2 MONTH)
  AND orderDate <= CURDATE();
```

This filters orders whose dates fall within the specified recent period, up to the current date.

26. String Functions

String functions manipulate text.

Function	**Purpose**
CONCAT()	Combines strings
UPPER()	Converts text to uppercase
LOWER()	Converts text to lowercase
LENGTH()	Returns byte length
CHAR_LENGTH()	Returns character count
SUBSTRING()	Extracts part of a string
TRIM()	Removes leading and trailing spaces
REPLACE()	Replaces matching text

Examples

```
SELECT CONCAT(firstName, ' ', lastName) AS FullName
FROM employees;
```

Combines first and last names.

```
SELECT UPPER(customerName)
FROM customers;
```

Converts names to uppercase.

SELECT LOWER(customerName)

FROM customers;

Converts names to lowercase.

SELECT LENGTH('Hello');

Returns the byte length of the string.

SELECT CHAR_LENGTH('Hello');

Returns the number of characters.

SELECT SUBSTRING('Database', 1, 4);

Returns Data.

SELECT TRIM(' Hello ');

Removes leading and trailing spaces.

SELECT REPLACE('SQL Basics', 'Basics', 'Practice');

Returns SQL Practice.

Difference: LENGTH() measures bytes, while CHAR_LENGTH() measures characters. They can differ for multibyte character sets.

27. Numeric Functions

Numeric functions perform mathematical calculations.

Function	**Purpose**
ROUND()	Rounds a number
CEIL()	Rounds upward to an integer
FLOOR()	Rounds downward to an integer
ABS()	Returns the absolute value
MOD()	Returns the remainder
POWER()	Raises a value to a power
SQRT()	Calculates the square root

Examples

SELECT ROUND(123.456, 2);

Returns 123.46.

SELECT CEIL(12.2);

Returns 13.

SELECT FLOOR(12.8);

Returns 12.

SELECT ABS(-25);

Returns 25.

SELECT MOD(10, 3);

Returns 1.

SELECT POWER(2, 3);

Returns 8.

SELECT SQRT(25);

Returns 5.

Group 9 — Set Operations

28. UNION

Definition: Combines rows from two query results and removes duplicate rows.

```
SELECT country
FROM customers
WHERE country = 'USA'

UNION

SELECT country
FROM customers
WHERE country = 'France';
```

The results of both queries are combined, with duplicate rows removed.

29. UNION ALL

Definition: Combines rows from two query results without removing duplicates.

SELECT country

InterviewQuestions

FROM customers

WHERE country = 'USA'

UNION ALL

SELECT country

FROM customers

WHERE country = 'France';

Unlike UNION, UNION ALL retains duplicate rows.

30. EXCEPT

Definition: Returns rows present in the first query result but not in the second query result.

SELECT customerNumber

FROM customers

EXCEPT

SELECT customerNumber

FROM orders;

Returns customer numbers that do not appear among the order records.

31. INTERSECT

Definition: Returns rows common to both query results.

SELECT customerNumber

FROM customers

INTERSECT

SELECT customerNumber

FROM orders;

Returns customer numbers present in both results.

Important comparison

Operator	What it returns
UNION	Combined results without duplicates
UNION ALL	Combined results including duplicates
EXCEPT	Rows in the first result but not the second
INTERSECT	Rows common to both results

Requirements: Set-operation queries should return the same number of columns in corresponding positions, with compatible data types.

MySQL version note: MySQL added INTERSECT and EXCEPT in version 8.0.31. Older versions may reject these operators.

dev.mysql.com

+2

Group 10 — Views, Stored Procedures and Triggers

32. VIEW

Definition: A view is a named, saved query that can be accessed like a virtual table. It generally does not store a separate copy of the query results.

Create a view

```
CREATE VIEW customer_payments AS
SELECT
  c.customerNumber,
  c.customerName,
  SUM(p.amount) AS TotalPayments
FROM customers AS c
JOIN payments AS p
  ON c.customerNumber = p.customerNumber
```

```
GROUP BY
  c.customerNumber,
  c.customerName;
```

Use the view

```
SELECT *
FROM customer_payments;
```

Drop the view

```
DROP VIEW IF EXISTS customer_payments;
```

Use case: Save a frequently used query and reuse it without rewriting the entire SQL statement.

33. Stored Procedure

Definition: A stored procedure is a named collection of SQL statements stored in the database and executed using CALL.

Example

```
DROP PROCEDURE IF EXISTS country_wise_customers;


DELIMITER $$


CREATE PROCEDURE country_wise_customers(
  IN p_country VARCHAR(30)
)
BEGIN
  SELECT *
  FROM customers
  WHERE country = p_country;
END $$


DELIMITER ;
```

```
CALL country_wise_customers('USA');
```

Explanation

- CREATE PROCEDURE defines the procedure.
- IN p_country accepts an input value.
- BEGIN and END enclose the procedure's SQL statements.
- DELIMITER changes the delimiter while defining the procedure.
- CALL executes it.

You can change the supplied country without recreating the procedure:

```
CALL country_wise_customers('France');
```

34. TRIGGER

Definition: A trigger is SQL logic that automatically executes when a specified event occurs on a table, such as an insert, update or delete.

Example — Check Credit Limit Before Insert

```
DELIMITER $$

CREATE TRIGGER check_credit_limit
BEFORE INSERT ON customers
FOR EACH ROW
BEGIN
  IF NEW.creditLimit < 0 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Credit limit cannot be negative';
  END IF;
END $$

DELIMITER ;
```

Explanation

- BEFORE INSERT means the trigger runs before a row is inserted.

- FOR EACH ROW means it runs for each affected row.
- NEW.creditLimit refers to the credit limit in the incoming row.
- SIGNAL raises an error if the value is negative.

This example assumes the customers table contains a creditLimit column.

Use case: Apply automatic checks or actions when table data changes.

Group 11 — Window Functions

35. Window Functions

Definition: A window function calculates a value across related rows while retaining individual rows in the result.

Unlike a typical GROUP BY aggregate, it can produce a calculation for each row without combining the entire group into one row.

Common syntax

```
FUNCTION_NAME() OVER (
  PARTITION BY column
  ORDER BY column
)
```

- OVER() defines the window.
- PARTITION BY divides the rows into groups for the calculation.
- ORDER BY determines the order within each window.

A. ROW_NUMBER()

Assigns a sequential number to each row.

```
SELECT
  employeeNumber,
  firstName,
  ROW_NUMBER() OVER (
    ORDER BY employeeNumber
  ) AS RowNum
FROM employees;
```

B. RANK()

Assigns ranks and leaves gaps after tied values.

For example, ranking values of 100, 100, 90 gives ranks 1, 1, 3.

C. DENSE_RANK()

Assigns ranks without gaps after tied values.

For values of 100, 100, 90, the ranks are 1, 1, 2.

D. NTILE()

Divides ordered rows into a specified number of buckets.

NTILE(4) OVER (ORDER BY creditLimit DESC)

Divides the ordered records into four groups as evenly as possible.

E. LAG()

Retrieves a value from a preceding row in the window.

```
LAG(amount) OVER (
  PARTITION BY customerNumber
  ORDER BY paymentDate
)
```

Can retrieve a customer's previous payment amount.

F. LEAD()

Retrieves a value from a following row in the window.

```
LEAD(amount) OVER (
  PARTITION BY customerNumber
  ORDER BY paymentDate
)
```

Can retrieve a customer's next payment amount.

G. FIRST_VALUE()

Returns the first value in the defined window frame.

```
FIRST_VALUE(amount) OVER (
  PARTITION BY customerNumber
  ORDER BY paymentDate
```

)

Can return the first payment amount in the ordered window.

H. LAST_VALUE()

Returns the last value in the defined window frame.

```
LAST_VALUE(amount) OVER (
  PARTITION BY customerNumber
  ORDER BY paymentDate
  ROWS BETWEEN UNBOUNDED PRECEDING
      AND UNBOUNDED FOLLOWING
)
```

The explicit frame lets the function consider the entire partition, so it can return the last payment amount in that window.

GROUP BY vs. Window Functions

GROUP BY	**Window functions**
Produces grouped results	Retains individual rows
Often produces one row per group	Can produce a calculation for every row
Example: one total per customer	Example: each payment with a running total

Group 12 — Subquery

36. Subquery

Definition: A subquery is a SQL query written inside another SQL statement.

The inner query provides a value or result set that the outer query uses.

Example — Customers Above Average Credit Limit

```
SELECT
  customerNumber,
```

```
  customerName,
  creditLimit
FROM customers
WHERE creditLimit > (
  SELECT AVG(creditLimit)
  FROM customers
);
```

Explanation

The inner query:

```
SELECT AVG(creditLimit)
FROM customers;
```

calculates the average credit limit.

The outer query returns customers whose credit limit is greater than that average.

Subqueries can appear in SELECT, FROM or WHERE clauses and may return one value or multiple rows, depending on their purpose.

Group 13 — CASE Statement

37. CASE Statement

Definition: CASE evaluates conditions and returns a value based on the first matching condition.

It is similar to an IF / ELSE IF / ELSE decision structure.

Example — Categorize Customers by Credit Limit

```
SELECT
  customerName,
  creditLimit,
  CASE
    WHEN creditLimit >= 100000 THEN 'High'
    WHEN creditLimit >= 50000 THEN 'Medium'
    ELSE 'Low'
  END AS CreditCategory
```

FROM customers;

Explanation

- If the credit limit is at least 100,000, the result is High.
- Otherwise, if it is at least 50,000, the result is Medium.
- Otherwise, the result is Low.

Conditions are checked from top to bottom. The first matching condition determines the result.

Use case: Categorize, label or transform data inside a query.

Group 14 — Normalization

38. Database Normalization

Definition: Database normalization is the process of organizing tables and their relationships to reduce unnecessary duplication and improve data consistency.

It helps prevent insertion, update and deletion anomalies.

A. First Normal Form — 1NF

Definition: A table is in 1NF when each field contains a single value rather than a list of multiple values, and rows can be distinguished by a key.

Poor design:

customerNumber	customerName	phoneNumbers
103	Store A	1111, 2222

The phoneNumbers field contains multiple phone numbers in one cell.

A better design separates the phone numbers into individual rows:

customerNumber	phoneNumber
103	1111
103	2222

Each field now contains a single phone number.

B. Second Normal Form — 2NF

Definition: A table is in 2NF when it is in 1NF and every non-key attribute depends on the whole of each candidate key, not just part of a composite key.

This is particularly important when the key contains more than one column.

Example:

Suppose an order-detail table uses (orderNumber, productCode) as its composite key, but productName depends only on productCode.

The product name depends on only part of the composite key.

A common solution is to store product details in the products table and use productCode to reference the product.

C. Third Normal Form — 3NF

Definition: A table is in 3NF when it is in 2NF and its non-key attributes do not depend transitively on candidate keys.

In simple terms, non-key attributes should depend on the key, the whole key, and not on another non-key attribute.

Example:

Suppose an employee table stores:

- employeeNumber
- officeCode
- officeCity

If the city is determined by the office code, then officeCity depends on officeCode rather than directly on the employee's key.

A better design stores office details in a separate offices table and keeps officeCode in employees as a reference.

Normalization comparison

Normal form	**Main rule**
1NF	Each field contains a single value
2NF	No non-key attribute depends on only part of a candidate key
3NF	No transitive dependency of non-key attributes on a candidate key

Final Revision — Complete Topic Map

Topic	**What to remember**
DBMS	Software for managing databases
RDBMS	DBMS based on related tables

SQL	Language used to work with relational databases
DDL	Defines or changes database structures
DML	Inserts, updates and deletes records
DQL	Retrieves records using SELECT
CRUD	Create, Read, Update, Delete
Data types	Define what values a column can store
Constraints	Rules that protect data integrity
Primary key	Uniquely identifies a row
WHERE	Filters individual rows
JOIN	Connects related tables
GROUP BY	Groups rows for calculations
HAVING	Filters grouped results
SUM()	Calculates totals
COUNT()	Counts rows or non-NULL values
AVG()	Calculates averages
Date functions	Manipulate and calculate with dates and times
String functions	Manipulate text
Numeric functions	Perform mathematical operations
Set operations	Combine or compare query results
VIEW	Saves a query as a virtual table
Stored procedure	Saved SQL logic executed using CALL
Trigger	Automatically executes on a table event
Window functions	Calculate across related rows without collapsing them
Subquery	Query nested inside another query
CASE	Returns values according to conditions
Normalization	Organizes tables to reduce redundancy

Your study rule: First understand the definition, then the syntax, then the example. For SQL questions, identify what result is needed before choosing which clauses and functions to use.

16.1. DBMS & SQL Topic Index

Group 1 — Database Fundamentals

1. DBMS — Database Management System
Definition: Software used to create, store, retrieve and manage data.

2. RDBMS — Relational Database Management System
Definition: A type of DBMS that organizes data into related tables.

3. Types of DBMS

4. Difference Between DBMS and RDBMS

5. Use Cases of DBMS

Group 2 — SQL Fundamentals

6. SQL — Structured Query Language
Definition: A language used to define, retrieve and manipulate data in relational databases.

7. Types of SQL Commands

- DDL
- DML
- DQL
- DCL
- TCL

8. CRUD Operations

- C — Create: INSERT
- R — Read: SELECT
- U — Update: UPDATE
- D — Delete: DELETE

Group 3 — SQL Commands

9. DDL — Data Definition Language

- CREATE

- DROP
- ALTER
- TRUNCATE

10. DML — Data Manipulation Language
    - INSERT
    - UPDATE
    - DELETE
11. DQL — Data Query Language
    - SELECT

Group 4 — Data Types

12. SQL Data Types
    - Numeric Data Types
    - String Data Types
    - Date and Time Data Types

Group 5 — SELECT Query Clauses

13. SELECT
14. FROM
15. WHERE
16. JOIN
17. DISTINCT
18. GROUP BY
19. HAVING
20. ORDER BY
21. LIMIT

Group 6 — Constraints

22. Constraints — Rules used to maintain data accuracy and integrity.
    - UNIQUE
    - NOT NULL

- PRIMARY KEY
- DEFAULT

Group 7 — Keys

23. Database Keys

- Primary Key

Group 8 — SQL Functions

24. Aggregate Functions

- COUNT()
- SUM()
- AVG()
- MIN()
- MAX()

25. Date Functions

- CURDATE()
- NOW()
- YEAR()
- MONTH()
- DAY()
- DATE_ADD()
- DATE_SUB()
- DATEDIFF()
- DATE_FORMAT()

26. String Functions

- CONCAT()
- UPPER()
- LOWER()
- LENGTH()
- CHAR_LENGTH()

- SUBSTRING()
- TRIM()
- REPLACE()

27. Numeric Functions

- ROUND()
- CEIL()
- FLOOR()
- ABS()
- MOD()
- POWER()
- SQRT()

Group 9 — Set Operations

28. UNION

29. UNION ALL

30. EXCEPT

31. INTERSECT

Group 10 — Views, Stored Procedures and Triggers

32. VIEW
Definition: A saved query that can be accessed like a virtual table.

33. Stored Procedure
Definition: A named collection of SQL statements stored in the database and executed using CALL.

34. TRIGGER
Definition: SQL logic that automatically executes when a specified table event occurs.

Group 11 — Window Functions

35. Window Functions

- ROW_NUMBER()
- RANK()
- DENSE_RANK()

- NTILE()
- LAG()
- LEAD()
- FIRST_VALUE()
- LAST_VALUE()

Group 12 — Subquery

36. Subquery
Definition: A query written inside another SQL query.

Group 13 — CASE Statement

37. CASE Statement
Definition: Evaluates conditions and returns a value based on the first matching condition.

Group 14 — Normalization

38. Database Normalization
Definition: Organizing database tables to reduce unnecessary duplication and improve data consistency.

- First Normal Form — 1NF
- Second Normal Form — 2NF
- Third Normal Form — 3NF

Bas, re baba. Ye tumhari di hui list ka organized version hai—extra topics ke groups nahi jode. Ab ise apne External Brain ke topic index ki tarah use kar sakte ho. ❤️

16.2. Keys and Numeric Functions

1. Keys

- Key: A column or set of columns used to identify records or establish relationships between tables.
- Super Key: Any column or combination of columns that can uniquely identify a row.
- Candidate Key: A minimal super key.
- Primary Key: A candidate key selected to uniquely identify a record.
- Alternative Key: A candidate key that was not selected as the primary key.

- Foreign Key: A column that references a key, commonly the primary key of another table.
- Composite Key: A key made using two or more columns together.

2. Numerical Functions

Function	Use
ROUND()	Rounds a number.
CEIL()	Rounds up.
FLOOR()	Rounds down.
ABS()	Returns the absolute value.
MOD()	Finds the remainder.
POWER()	Calculates the power of a number.
SQRT()	Calculates the square root.
SIGN()	Returns +1, -1, or 0.
RAND()	Returns a random value.

16.3. Keys and Numerical Functions(detailed)

Part 1: Keys in DBMS

1. What is a Key?

A key is a column or a combination of columns used to identify records in a table or establish relationships between tables.

Imagine a college database with a Students table:

StudentID	Name	Email
101	Ahad	ahad@gmail.com
102	Rahul	rahul@gmail.com
103	Sana	sana@gmail.com

Here, StudentID can identify each student because every student has a different ID.

However, Name may not uniquely identify a student because two students can have the same name.

Main purpose of keys:

- Identify a particular record.
- Maintain uniqueness where required.
- Connect related tables.
- Help maintain data integrity.

There are different types of keys, and each has a specific purpose.

2. Super Key

A super key is any column or combination of columns that can uniquely identify a row in a table.

Consider this table:

StudentID	Name	Email
101	Ahad	ahad@gmail.com
102	Rahul	rahul@gmail.com
103	Sana	sana@gmail.com

Assume StudentID and Email are both unique.

Examples of super keys include:

Super key	Why is it a super key?
StudentID	Each student has a unique ID.
Email	Each student has a unique email.
StudentID, Name	The combination is unique because StudentID is already unique.
StudentID, Email	The combination is also unique.

Important concept: A super key can contain unnecessary columns.

For example, StudentID is already enough to identify a student. Adding Name does not improve its uniqueness.

That is why we have another concept called the candidate key.

3. Candidate Key

A candidate key is a minimal super key.

The word minimal means that no column can be removed from the key without losing its ability to uniquely identify a row.

Using the same Students table:

- StudentID is a candidate key.
- Email is a candidate key.
- StudentID + Name is not a candidate key because Name is unnecessary.
- StudentID + Email is not a candidate key because either column alone is sufficient.

Therefore, if both StudentID and Email are individually unique, both can be candidate keys.

Remember the relationship:

Every candidate key is a super key, but every super key is not necessarily a candidate key.

Think of it this way

Super Key

Any unique identifying column or combination

Remove unnecessary columns

Candidate Key

A minimal super key

4. Primary Key

A primary key is a candidate key selected to uniquely identify each record in a table.

Suppose we have two candidate keys:

- StudentID
- Email

We decide to use StudentID as the primary key.

StudentID (Primary Key)	**Name**	**Email**
101	Ahad	ahad@gmail.com
102	Rahul	rahul@gmail.com
103	Sana	sana@gmail.com

The primary key has two important rules:

- Unique: Its value cannot be duplicated.

- NOT NULL: It cannot be missing.

A table has only one primary-key constraint, although that primary key can consist of multiple columns.

Practical example: A college can use StudentID to find the exact student even if multiple students have the same name.

5. Alternative Key

An alternative key, also called an alternate key, is a candidate key that was not selected as the primary key.

Suppose:

- StudentID is the primary key.
- Email is another candidate key.

Then Email is an alternative key because it could uniquely identify a student but was not selected as the primary key.

Key	**Role**
StudentID	Selected as the primary key
Email	Alternative key

An alternative key must still be a valid candidate key. A non-unique column such as Name does not qualify.

6. Foreign Key

A foreign key is a column or combination of columns that references a key in another table, commonly its primary key.

Imagine two tables.

Customers table

customerNumber	**customerName**
1	Ahad
2	Rahul
3	Sana

Orders table

orderNumber	**customerNumber**

501	1
502	2
503	1

In the Customers table, customerNumber identifies each customer.

In the Orders table, customerNumber tells us which customer placed an order.

The relationship is:

- Customers.customerNumber — primary key.
- Orders.customerNumber — foreign key.

For example, order 501 belongs to customer 1, which is Ahad.

A foreign key helps maintain referential integrity. For example, an order should not refer to a customer who does not exist in the referenced table, subject to the database's constraints and rules.

Important: A foreign key does not have to be unique. A customer may place many orders, so the same customerNumber can appear in multiple rows of the Orders table.

7. Composite Key

A composite key is a key made using two or more columns together.

Suppose an OrderDetails table contains:

orderNumber	**productCode**	**quantity**
501	P101	2
501	P102	1
502	P101	4

Here:

- orderNumber alone is not unique because one order can contain multiple products.
- productCode alone is not unique because the same product can appear in multiple orders.
- The combination (orderNumber, productCode) identifies each order-product entry.

Therefore, (orderNumber, productCode) can be a composite primary key.

Important: A composite key is not a separate alternative to all the other key types. For example, a composite key can also be a primary key or a candidate key.

Part 2: Numerical Functions in SQL

Numerical functions perform calculations on numeric values. The following examples use SQL's SELECT statement so that you can see the result of each function.

1. ROUND() — Round a Number

Meaning: ROUND() rounds a number to a specified number of decimal places.

Syntax

ROUND(number, decimal_places)

Example 1: Round to two decimal places

SELECT ROUND(12.567, 2) AS Result;

Output:

Result
12.57

Why? The number is 12.567. We keep two decimal places, and the next digit is 7. Since 7 is at least 5, the second decimal digit increases by one.

Example 2: Round to a whole number

SELECT ROUND(12.4) AS Result;

Output:

Result
12

The decimal part is less than 0.5, so the number rounds to 12.

SELECT ROUND(12.8) AS Result;

Output:

Result
13

The decimal part is at least 0.5, so the number rounds to 13.

Use case: Rounding sales, prices, percentages, and averages.

2. CEIL() — Round Up

Meaning: CEIL() returns the smallest integer that is greater than or equal to the given number.

Think of it as moving up to the next integer when necessary.

Syntax

CEIL(number)

Example

SELECT CEIL(4.2) AS Result;

Output:

Result
5

Why? The integers greater than or equal to 4.2 begin at 5. Therefore, the result is 5.

Another example:

SELECT CEIL(4.0) AS Result;

Output:

Result
4

If the number is already an integer, it stays unchanged.

Use case: Calculating the number of whole boxes or trips needed when a partial box or trip is not sufficient.

3. FLOOR() — Round Down

Meaning: FLOOR() returns the greatest integer that is less than or equal to the given number.

Think of it as moving down to the previous integer when necessary.

Syntax

FLOOR(number)

Example

SELECT FLOOR(4.8) AS Result;

Output:

Result

4

Why? The greatest integer that does not exceed 4.8 is 4.

Another example:

SELECT FLOOR(4.0) AS Result;

Output:

Result

4

The value remains unchanged because it is already an integer.

Important difference between CEIL() and FLOOR(): They work with the mathematical direction of the number line, not simply by changing the decimal part.

For example, CEIL(-4.2) returns -4, while FLOOR(-4.2) returns -5.

4. ABS() — Absolute Value

Meaning: ABS() returns the absolute value of a number. It removes the negative sign from a negative number.

Syntax

ABS(number)

Example 1

SELECT ABS(-25) AS Result;

Output:

Result

25

The absolute value of -25 is 25.

Example 2

SELECT ABS(25) AS Result;

Output:

Result

25

A positive number remains positive.

Example 3

SELECT ABS(0) AS Result;

Output:

Result

0

Use case: Finding the magnitude of a difference without considering whether the original difference was positive or negative.

For example, if the difference between two prices is -50, then ABS(-50) returns 50.

5. MOD() — Find the Remainder

Meaning: MOD() returns the remainder after dividing one number by another.

Syntax

MOD(dividend, divisor)

- Dividend: The number being divided.
- Divisor: The number by which it is divided.

Example 1

SELECT MOD(17, 5) AS Result;

Output:

Result

2

Why?

\[17 = (5 \times 3) + 2 \]

Five fits into seventeen three times, leaving a remainder of 2.

Example 2

SELECT MOD(20, 5) AS Result;

Output:

Result
0

Twenty is exactly divisible by five, so the remainder is zero.

Example 3

SELECT MOD(14, 2) AS Result;

Output:

Result
0

This is useful for checking whether a number is even: an even number leaves a remainder of zero when divided by two.

Use case: Checking even or odd numbers, repeating patterns, and divisibility.

6. POWER() — Calculate a Power

Meaning: POWER() raises a number to a specified exponent.

Syntax

POWER(base, exponent)

- Base: The number being multiplied repeatedly.
- Exponent: How many times the base appears as a factor, for a positive whole-number exponent.

Example 1

SELECT POWER(2, 3) AS Result;

Output:

Result
8

Why?

\[2^3 = 2 \times 2 \times 2 = 8 \]

Example 2

SELECT POWER(5, 2) AS Result;

Output:

Result
25

Why?

\[5^2 = 5 \times 5 = 25 \]

Example 3

SELECT POWER(10, 3) AS Result;

Output:

Result
1000

Use case: Calculating squares, cubes, and other mathematical powers.

7. SQRT() — Square Root

Meaning: SQRT() returns the non-negative square root of a number.

The square root of a number is a value that, when multiplied by itself, gives the original number.

Syntax

SQRT(number)

Example 1

SELECT SQRT(81) AS Result;

Output:

Result
9

Why?

\[9 \times 9 = 81 \]

Therefore, the square root of 81 is 9.

Example 2

SELECT SQRT(25) AS Result;

Output:

Result

5

Because \(5 \times 5 = 25\).

Example 3

SELECT SQRT(2) AS Result;

Output:

Result

Approximately 1.4142

The result is an approximate decimal because the square root of 2 is not a whole number.

Use case: Mathematical calculations involving squares and distances.

8. SIGN() — Identify the Sign

Meaning: SIGN() tells you whether a number is positive, negative, or zero.

Syntax

SIGN(number)

It returns:

- 1 when the number is positive.
- -1 when the number is negative.
- 0 when the number is zero.

Example 1

SELECT SIGN(25) AS Result;

Output:

Result

1

The number is positive.

Example 2

SELECT SIGN(-25) AS Result;

Output:

Result

-1

The number is negative.

Example 3

SELECT SIGN(0) AS Result;

Output:

Result

0

The number is zero.

Important: SIGN() does not return the original number. It returns only its sign indicator.

For example, SIGN(500) returns 1, not 500.

Use case: Identifying whether a calculated difference represents an increase, a decrease, or no change.

9. RAND() — Generate a Random Value

Meaning: RAND() generates a pseudo-random decimal value greater than or equal to 0 and less than 1.

Syntax

RAND()

Example

SELECT RAND() AS Result;

Possible output:

Result

0.7352

Your output may be different because the value is generated randomly.

For example, one execution might return 0.7352, while another might return 0.1826.

Example: Generate a random integer from 1 to 10

```
SELECT FLOOR(RAND() * 10) + 1 AS RandomNumber;
```

Possible output:

RandomNumber

7

How does it work?

1. RAND() generates a value from 0 up to, but not including, 1.
2. Multiplying by 10 gives a value from 0 up to, but not including, 10.
3. FLOOR() converts it to an integer from 0 to 9.
4. Adding 1 produces an integer from 1 to 10.

Use case: Generating sample values or random selections for suitable tasks. Random values are not guaranteed to be unique.

Final revision table

Function	Main purpose	Example	Result
ROUND()	Round to specified decimal places	ROUND(12.567, 2)	12.57
CEIL()	Return the next integer upward	CEIL(4.2)	5
FLOOR()	Return the integer downward	FLOOR(4.8)	4
ABS()	Return absolute value	ABS(-25)	25
MOD()	Return remainder	MOD(17, 5)	2
POWER()	Raise a number to a power	POWER(2, 3)	8
SQRT()	Calculate square root	SQRT(81)	9
SIGN()	Identify positive, negative, or zero	SIGN(-25)	-1
RAND()	Generate a random decimal	RAND()	Varies

The easiest way to remember them:

ROUND = round, CEIL = upward, FLOOR = downward, ABS = remove the negative sign, MOD = remainder, POWER = exponent, SQRT = square root, SIGN = positive/negative/zero, and RAND = random value.

16.4. Date Functions Summary

1. Current Date and Time

Function	Use	Example
CURRENT_DATE()	Returns the current date.	SELECT CURRENT_DATE();
CURRENT_TIME()	Returns the current time.	SELECT CURRENT_TIME();
CURRENT_TIMESTAMP()	Returns the current date and time.	SELECT CURRENT_TIMESTAMP();
NOW()	Returns the current date and time.	SELECT NOW();

2. EXTRACT() — Extract a Date Part

Extracts a specific part of a date, such as the month, year, or day.

```
SELECT EXTRACT(MONTH FROM orderDate)
FROM orders;

SELECT EXTRACT(YEAR FROM orderDate)
FROM orders;

SELECT EXTRACT(DAY FROM orderDate)
FROM orders;
```

3. DATE_FORMAT() — Format a Date

Converts a date or datetime value into a formatted string.

```
SELECT DATE_FORMAT(orderDate, '%d-%m-%y')
FROM orders;

SELECT DATE_FORMAT(orderDate, '%b-%y')
FROM orders;

SELECT DATE_FORMAT(NOW(), '%b-%y');
```

Common format specifiers

Specifier	Meaning

%d	Day as a two-digit number
%D	Day with an ordinal suffix, such as 1st
%M	Full month name
%b	Abbreviated month name
%m	Month as a two-digit number
%Y	Four-digit year
%y	Two-digit year
%H	Hour (24-hour format)
%i	Minutes
%s	Seconds

4. DATEDIFF() — Difference Between Dates

Returns the difference between two dates in days.

```
SELECT DATEDIFF(NOW(), orderDate)
FROM orders;
```

```
SELECT DATEDIFF(shippedDate, orderDate)
FROM orders;
```

- DATEDIFF(NOW(), orderDate) → Days between the order date and current date.
- DATEDIFF(shippedDate, orderDate) → Days between the order date and shipping date.

5. DATE_ADD() — Add an Interval

Adds a specified time interval to a date.

```
SELECT DATE_ADD(shippedDate, INTERVAL 1 MONTH)
FROM orders;
```

Adds one month to the shipping date.

6. DATE_SUB() — Subtract an Interval

Subtracts a specified time interval from a date.

```
SELECT DATE_SUB(shippedDate, INTERVAL 2 MONTH)
```

InterviewQuestions

FROM orders;

SELECT DATE_SUB('2020-08-12', INTERVAL 2 MONTH);

The second example returns 2020-06-12.

7. Individual Date Functions

Function	Use	Example
YEAR()	Extracts the year.	YEAR(orderDate)
MONTH()	Extracts the month number.	MONTH(orderDate)
DAY()	Extracts the day of the month.	DAY(orderDate)
DAYNAME()	Returns the weekday name.	DAYNAME(orderDate)
MONTHNAME()	Returns the month name.	MONTHNAME(orderDate)
QUARTER()	Returns the quarter, from 1 to 4.	QUARTER(orderDate)

Remember: EXTRACT() selects a date part, DATE_FORMAT() changes how the date is displayed, DATEDIFF() calculates the difference in days, DATE_ADD() adds time, and DATE_SUB() subtracts time.

16.5. Date Functions — Full Detailed Notes

Date functions in SQL are used to retrieve the current date and time, extract parts of a date, change the display format, calculate the difference between dates, and add or subtract time intervals.

These examples use MySQL and the orders table.

1. Current Date and Time Functions

These functions return the current date, current time, or both.

1.1 CURRENT_DATE()

Meaning: Returns the current date, without the time.

Syntax and example

SELECT CURRENT_DATE();

Possible output:

CURRENT_DATE()

2026-10-24

Explanation: The output contains the year, month, and day in YYYY-MM-DD format. It does not include hours, minutes, or seconds.

Use: When you need today's date for a query or calculation.

1.2 CURRENT_TIME()

Meaning: Returns the current time, without the date.

SELECT CURRENT_TIME();

Possible output:

CURRENT_TIME()

11:35:20

Explanation: The output contains hours, minutes, and seconds. The actual result depends on the time when you execute the query and the database session's time zone.

Use: When you need the current time.

1.3 CURRENT_TIMESTAMP()

Meaning: Returns the current date and time together.

SELECT CURRENT_TIMESTAMP();

Possible output:

CURRENT_TIMESTAMP()

2026-10-24 11:35:20

Explanation: It returns the date followed by the time.

- Date: 2026-10-24
- Time: 11:35:20

Use: When you need both the current date and time.

1.4 NOW()

Meaning: Returns the current date and time, similar to CURRENT_TIMESTAMP() in MySQL.

SELECT NOW();

Possible output:

NOW()
2026-10-24 11:35:20

The exact result depends on when you execute the query.

Remember: CURRENT_DATE() gives the date, CURRENT_TIME() gives the time, and both CURRENT_TIMESTAMP() and NOW() give the date and time.

2. EXTRACT() — Extract a Date Part

Meaning: EXTRACT() retrieves a specific part of a date, such as the month, year, or day.

Syntax

EXTRACT(unit FROM date)

- unit specifies which part you want.
- date is the date from which you want that part.

Suppose an order has this date:

2024-03-15

The year is 2024, the month is 03, and the day is 15.

2.1 Extract the Month

```
SELECT EXTRACT(MONTH FROM orderDate)
FROM orders;
```

Possible result for an order dated 2024-03-15:

EXTRACT(MONTH FROM orderDate)
3

Explanation: MONTH tells SQL to return the month number. March is month 3.

2.2 Extract the Year

```
SELECT EXTRACT(YEAR FROM orderDate)
FROM orders;
```

Output for 2024-03-15:

EXTRACT(YEAR FROM orderDate)

2024

Explanation: YEAR returns the four-digit year.

2.3 Extract the Day

SELECT EXTRACT(DAY FROM orderDate)

FROM orders;

Output for 2024-03-15:

EXTRACT(DAY FROM orderDate)

15

Explanation: DAY returns the day of the month, not the day of the week.

Remember: EXTRACT() selects a particular part of a date. It does not return the entire date.

3. DATE_FORMAT() — Format a Date

Meaning: DATE_FORMAT() converts a date or datetime into a formatted string.

You use it when you want to control how a date appears in the output.

Syntax

DATE_FORMAT(date, format)

- date is the date or datetime value.
- format specifies how you want the date displayed.

Suppose:

orderDate = 2024-03-15

3.1 Format the Date as Day-Month-Year

SELECT DATE_FORMAT(orderDate, '%d-%m-%y')

FROM orders;

Output:

DATE_FORMAT(orderDate, '%d-%m-%y')

15-03-24

Explanation of the format:

- %d = two-digit day.
- %m = two-digit month.
- %y = two-digit year.

Therefore, 2024-03-15 becomes 15-03-24.

3.2 Format the Date as Abbreviated Month-Year

```
SELECT DATE_FORMAT(orderDate, '%b-%y')
FROM orders;
```

Output:

DATE_FORMAT(orderDate, '%b-%y')
Mar-24

Explanation:

- %b = abbreviated month name, such as Mar.
- %y = two-digit year, such as 24.

The complete date is displayed only as month and year.

3.3 Format the Current Date and Time as Month-Year

```
SELECT DATE_FORMAT(NOW(), '%b-%y');
```

Possible output:

DATE_FORMAT(NOW(), '%b-%y')
Oct-26

Explanation:

1. NOW() provides the current date and time.
2. DATE_FORMAT() changes its displayed format.
3. %b-%y displays the abbreviated month and two-digit year.

The output excludes the day and time because the format does not request them.

Common DATE_FORMAT() Specifiers

Specifier	Meaning	Example
%d	Day as a two-digit number	05

%D	Day with an ordinal suffix	5th
%M	Full month name	March
%b	Abbreviated month name	Mar
%m	Month as a two-digit number	03
%Y	Four-digit year	2024
%y	Two-digit year	24
%H	Hour in 24-hour format	14
%i	Minutes	35
%s	Seconds	20

Important difference:

- %M means the full month name.
- %m means the numeric month.
- %Y means the four-digit year.
- %y means the two-digit year.
- %M and %m are different because SQL format specifiers are case-sensitive.

For example, for 2024-03-15 14:35:20, the format below:

```
SELECT DATE_FORMAT(
   '2024-03-15 14:35:20',
   '%d-%M-%Y %H:%i:%s'
);
```

returns:

15-March-2024 14:35:20

4. DATEDIFF() — Difference Between Dates

Meaning: DATEDIFF() calculates the difference between two dates in days.

Syntax

DATEDIFF(date1, date2)

The calculation is:

\[\text{DATEDIFF} = \text{date1} - \text{date2} \]

It returns the number of days. The time portions of datetime values are ignored.

4.1 Difference Between Today and the Order Date

SELECT DATEDIFF(NOW(), orderDate)

FROM orders;

Suppose:

- Current date = 2024-03-25
- Order date = 2024-03-15

Output:

DATEDIFF(NOW(), orderDate)
10

Explanation: Ten days have passed between the order date and the current date in this example.

This query calculates the age of each order in days.

4.2 Difference Between Shipping Date and Order Date

SELECT DATEDIFF(shippedDate, orderDate)

FROM orders;

Suppose:

- orderDate = 2024-03-15
- shippedDate = 2024-03-20

Output:

DATEDIFF(shippedDate, orderDate)
5

Explanation: The order was shipped five days after it was placed.

Remember the order of arguments: DATEDIFF(shippedDate, orderDate) calculates shipping date minus order date. Reversing the arguments reverses the sign of the result.

5. DATE_ADD() — Add a Time Interval

Meaning: DATE_ADD() adds a specified time interval to a date or datetime value.

Syntax

DATE_ADD(date, INTERVAL number unit)

- date is the starting date.
- number is the amount to add.
- unit specifies the unit, such as DAY, MONTH, or YEAR.

Example: Add One Month to the Shipping Date

SELECT DATE_ADD(shippedDate, INTERVAL 1 MONTH)

FROM orders;

Suppose:

shippedDate = 2024-03-20

Output:

DATE_ADD(shippedDate, INTERVAL 1 MONTH)

2024-04-20

Explanation:

1. Start with 2024-03-20.
2. Add one month.
3. The result is 2024-04-20.

The function calculates a new date for each row in orders; it does not permanently change the stored shippedDate value.

Use: Calculating a date one month after shipping, or a future date based on another date.

6. DATE_SUB() — Subtract a Time Interval

Meaning: DATE_SUB() subtracts a specified time interval from a date or datetime value.

Syntax

DATE_SUB(date, INTERVAL number unit)

The syntax is similar to DATE_ADD(), but it moves backwards in time.

6.1 Subtract Two Months from the Shipping Date

SELECT DATE_SUB(shippedDate, INTERVAL 2 MONTH)

FROM orders;

Suppose:

shippedDate = 2024-03-20

Output:

DATE_SUB(shippedDate, INTERVAL 2 MONTH)
2024-01-20

Explanation:

1. Start with 2024-03-20.
2. Go back two months.
3. The result is 2024-01-20.

6.2 Subtract Two Months from a Fixed Date

SELECT DATE_SUB('2020-08-12', INTERVAL 2 MONTH);

Output:

DATE_SUB('2020-08-12', INTERVAL 2 MONTH)
2020-06-12

Explanation:

- Starting date: August 12, 2020.
- Subtract two months.
- Result: June 12, 2020.

Here, the starting date is a fixed value rather than a column from a table.

Remember: DATE_ADD() moves a date forward, while DATE_SUB() moves a date backward. When the target month does not contain the original day number, MySQL adjusts the result to the last valid day of the target month.

7. Individual Date Functions

These functions retrieve a specific part or name associated with a date.

For the examples below, assume:

orderDate = 2024-03-15

7.1 YEAR()

Meaning: Returns the year from a date.

```
SELECT YEAR(orderDate)
FROM orders;
```

Output for the example date:

YEAR(orderDate)
2024

The output is a numeric year, not the complete date.

7.2 MONTH()

Meaning: Returns the month number, from 1 to 12.

```
SELECT MONTH(orderDate)
FROM orders;
```

Output:

MONTH(orderDate)
3

March is month number 3.

7.3 DAY()

Meaning: Returns the day of the month, from 1 to 31, depending on the month.

```
SELECT DAY(orderDate)
FROM orders;
```

Output:

DAY(orderDate)
15

It returns the day of the month, not the name of the weekday.

7.4 DAYNAME()

Meaning: Returns the weekday name associated with a date.

```
SELECT DAYNAME(orderDate)
FROM orders;
```

Output for 2024-03-15:

DAYNAME(orderDate)
Friday

The output is a name, such as Monday, Friday, or Sunday, rather than a numeric day.

7.5 MONTHNAME()

Meaning: Returns the full month name associated with a date.

```
SELECT MONTHNAME(orderDate)
FROM orders;
```

Output:

MONTHNAME(orderDate)
March

It returns the name of the month rather than its numeric value.

7.6 QUARTER()

Meaning: Returns the calendar quarter of the year, from 1 to 4.

```
SELECT QUARTER(orderDate)
FROM orders;
```

Output for 2024-03-15:

QUARTER(orderDate)
1

A calendar year has four quarters:

Quarter	Months
Q1	January, February, March
Q2	April, May, June
Q3	July, August, September
Q4	October, November, December

Since March falls in the first quarter, QUARTER('2024-03-15') returns 1.

8. Complete Revision Table

Function	Purpose	Example
CURRENT_DATE()	Returns the current date	SELECT CURRENT_DATE();
CURRENT_TIME()	Returns the current time	SELECT CURRENT_TIME();
CURRENT_TIMESTAMP()	Returns the current date and time	SELECT CURRENT_TIMESTAMP();
NOW()	Returns the current date and time	SELECT NOW();
EXTRACT()	Extracts a date part	EXTRACT(MONTH FROM orderDate)
DATE_FORMAT()	Formats the display of a date	DATE_FORMAT(orderDate, '%d-%m-%y')
DATEDIFF()	Returns the difference between dates in days	DATEDIFF(shippedDate, orderDate)
DATE_ADD()	Adds a time interval	DATE_ADD(shippedDate, INTERVAL 1 MONTH)
DATE_SUB()	Subtracts a time interval	DATE_SUB(shippedDate, INTERVAL 2 MONTH)
YEAR()	Returns the year	YEAR(orderDate)
MONTH()	Returns the month number	MONTH(orderDate)
DAY()	Returns the day of the month	DAY(orderDate)
DAYNAME()	Returns the weekday name	DAYNAME(orderDate)
MONTHNAME()	Returns the full month name	MONTHNAME(orderDate)
QUARTER()	Returns the quarter number	QUARTER(orderDate)

Final concept to remember

- Current date/time functions tell you what the current date and time are.
- EXTRACT() and individual date functions retrieve particular parts of a date.
- DATE_FORMAT() controls how the date is displayed.
- DATEDIFF() calculates the difference between two dates in days.
- DATE_ADD() and DATE_SUB() calculate dates by adding or subtracting time intervals.

16.6. Window Functions — Summary

1. Definition

Window functions perform calculations across a group of rows while keeping individual rows in the result.

2. Types of Window Functions

A. Ranking Functions

Function	**Use**
ROW_NUMBER()	Assigns a unique sequential number to each row.
RANK()	Assigns ranks; ties can cause gaps in ranking.
DENSE_RANK()	Assigns ranks; ties do not cause gaps.
NTILE()	Divides rows into a specified number of groups.

B. Value Functions

Function	**Use**
LAG()	Retrieves a value from a previous row.
LEAD()	Retrieves a value from a following row.
FIRST_VALUE()	Returns the first value in the window frame.
LAST_VALUE()	Returns the last value in the window frame.

C. Aggregation Functions

Function	**Use**
SUM()	Calculates the total.
MIN()	Returns the minimum value.

3. Example: Total Credit Limit Across All Customers

```
SELECT
  customerName,
  country,
  creditLimit,
  SUM(creditLimit) OVER()
FROM customers;
```

Explanation:

- customerName — Displays the customer name.
- country — Displays the customer's country.
- creditLimit — Displays the individual customer's credit limit.
- SUM(creditLimit) OVER() — Calculates the total credit limit across all customers while keeping each customer's row in the result.

Remember: GROUP BY combines rows into groups, whereas a window function calculates across rows while retaining individual rows.

16.7. Window Functions — Full Detailed Notes

1. What Are Window Functions?

Definition: Window functions perform calculations across a group of rows while keeping individual rows in the result.

Imagine the customers table contains these records:

customerName	country	creditLimit
Alpha Ltd	USA	5000
Beta Ltd	UK	4000
Gamma Ltd	USA	4000
Delta Ltd	India	3000
Epsilon Ltd	UK	2000

Suppose you want to display every customer's name and credit limit, along with the total credit limit of all customers.

A window function can calculate that total without combining all customers into one row.

```
SELECT
  customerName,
  country,
  creditLimit,
  SUM(creditLimit) OVER () AS TotalCreditLimit
FROM customers;
```

Conceptual output for the sample data:

customerName	country	creditLimit	TotalCreditLimit
Alpha Ltd	USA	5000	18000
Beta Ltd	UK	4000	18000
Gamma Ltd	USA	4000	18000
Delta Ltd	India	3000	18000
Epsilon Ltd	UK	2000	18000

Notice that the total, 18000, appears in every row. All five customer records remain in the output.

Understanding OVER()

The OVER() clause tells SQL to perform a window calculation.

SUM(creditLimit) OVER ()

- SUM(creditLimit) calculates the sum of credit limits.
- OVER() defines the window of rows used by the calculation.
- Empty parentheses mean the calculation considers all rows in the query's result set as one window, assuming no partition is specified.

Remember: A window function calculates across rows without collapsing those rows into a single result row.

2. Types of Window Functions

Your notes contain three categories:

A. Ranking Functions

ROW_NUMBER(), RANK(), DENSE_RANK(), NTILE()

Assign row numbers, ranks, or groups to rows.

B. Value Functions

LAG(), LEAD(), FIRST_VALUE(), LAST_VALUE()

Retrieve values from previous, following, first, or last rows in the window.

C. Aggregation Functions

SUM(), MIN()

Calculate totals or minimum values while preserving individual rows.

3. Ranking Functions

Ranking functions assign numbers, ranks, or group numbers to rows based on an ordering.

The ORDER BY clause inside OVER() determines the order in which rows are ranked.

We will use the following credit limits:

Customer	Credit limit
Alpha Ltd	5000
Beta Ltd	4000
Gamma Ltd	4000
Delta Ltd	3000
Epsilon Ltd	2000

3.1 ROW_NUMBER() — Assign a Unique Row Number

Definition: Assigns a unique sequential number to each row in the window.

Syntax

```
ROW_NUMBER() OVER (ORDER BY column_name)
```

Example

```
SELECT
  customerName,
  creditLimit,
  ROW_NUMBER() OVER (
    ORDER BY creditLimit DESC, customerNumber
```

```
  ) AS RowNum
FROM customers;
```

Conceptual output for the sample data:

Customer	Credit limit	RowNum
Alpha Ltd	5000	1
Beta Ltd	4000	2
Gamma Ltd	4000	3
Delta Ltd	3000	4
Epsilon Ltd	2000	5

Explanation

- ROW_NUMBER() assigns the first row number as 1.
- ORDER BY creditLimit DESC puts the highest credit limit first.
- customerNumber provides a tie-breaker if customers have equal credit limits.

Even though Beta Ltd and Gamma Ltd both have a credit limit of 4000, they receive different row numbers.

Use case: Numbering rows or selecting the first few records from an ordered result.

3.2 RANK() — Rank Rows with Gaps for Ties

Definition: Assigns ranks based on ordering. When rows have equal values, they receive the same rank, and a gap appears in the following ranks.

Example

```
SELECT
  customerName,
  creditLimit,
  RANK() OVER (
    ORDER BY creditLimit DESC
  ) AS CustomerRank
FROM customers;
```

Conceptual output:

Customer	Credit limit	CustomerRank
Alpha Ltd	5000	1
Beta Ltd	4000	2
Gamma Ltd	4000	2
Delta Ltd	3000	4
Epsilon Ltd	2000	5

Why does Delta Ltd receive rank 4?

Beta Ltd and Gamma Ltd share rank 2. The next rank is 4, because two rows occupy the first two ranking positions and the rank for position 3 is skipped.

The sequence is:

1, 2, 2, 4, 5

Use case: Ranking customers, employees, or products when equal values should receive the same rank.

3.3 DENSE_RANK() — Rank Without Gaps

Definition: Assigns the same rank to equal values, but does not skip the next rank.

Example

```
SELECT
  customerName,
  creditLimit,
  DENSE_RANK() OVER (
    ORDER BY creditLimit DESC
  ) AS DenseRank
FROM customers;
```

Conceptual output:

Customer	Credit limit	DenseRank
Alpha Ltd	5000	1
Beta Ltd	4000	2

Gamma Ltd	4000	2
Delta Ltd	3000	3
Epsilon Ltd	2000	4

The sequence is:

1, 2, 2, 3, 4

Why does Delta Ltd receive rank 3?

The two customers with credit limits of 4000 share rank 2. The next distinct credit limit gets rank 3, without a gap.

Use case: Assigning ranks based on distinct credit-limit values or sales amounts.

Difference Between the Three Ranking Functions

Credit limit	ROW_NUMBER()	RANK()	DENSE_RANK()
5000	1	1	1
4000	2	2	2
4000	3	2	2
3000	4	4	3
2000	5	5	4

- ROW_NUMBER() gives each row a distinct sequential number.
- RANK() gives ties the same rank and leaves gaps.
- DENSE_RANK() gives ties the same rank but does not leave gaps.

3.4 NTILE() — Divide Rows into Groups

Definition: Divides ordered rows into a specified number of approximately equal-sized groups, called buckets.

Syntax

NTILE(number_of_groups) OVER (ORDER BY column_name)

Example

```
SELECT
  customerName,
```

```
  creditLimit,
  NTILE(3) OVER (
    ORDER BY creditLimit DESC, customerNumber
  ) AS CreditGroup
FROM customers;
```

Conceptual output for five rows:

Customer	Credit limit	CreditGroup
Alpha Ltd	5000	1
Beta Ltd	4000	1
Gamma Ltd	4000	2
Delta Ltd	3000	2
Epsilon Ltd	2000	3

Here, NTILE(3) divides five rows into three groups.

- Group 1: first two rows.
- Group 2: next two rows.
- Group 3: final row.

When the rows cannot be divided equally, the groups may differ in size by one row. Earlier groups receive the extra rows.

Use case: Dividing customers into tiers or splitting ordered data into groups for analysis.

4. Value Functions

Value functions retrieve values from rows in the ordered window. They are useful for comparing one row with another.

For the examples below, imagine sales data ordered by year:

Year	Sales
2022	10000
2023	15000
2024	12000

2025 18000

4.1 LAG() — Get a Value from a Previous Row

Definition: Retrieves a value from an earlier row in the window.

Syntax

LAG(column_name, offset, default_value)

OVER (ORDER BY column_name)

- column_name is the value to retrieve.
- offset is how many rows backward to look.
- default_value is optional and is returned when there is no preceding row at that offset.

Example

```
SELECT
  year,
  sales,
  LAG(sales) OVER (ORDER BY year) AS PreviousSales
FROM yearlySales;
```

Conceptual output:

Year	Sales	PreviousSales
2022	10000	NULL
2023	15000	10000
2024	12000	15000
2025	18000	12000

Explanation

- In 2022, no earlier row exists, so PreviousSales is NULL.
- In 2023, the previous year's sales were 10000.
- In 2024, the previous year's sales were 15000.
- In 2025, the previous year's sales were 12000.

Use case: Comparing current sales with the previous year's sales.

4.2 LEAD() — Get a Value from a Following Row

Definition: Retrieves a value from a later row in the window.

Example

```
SELECT
  year,
  sales,
  LEAD(sales) OVER (ORDER BY year) AS NextSales
FROM yearlySales;
```

Conceptual output:

Year	Sales	NextSales
2022	10000	15000
2023	15000	12000
2024	12000	18000
2025	18000	NULL

Explanation

- For 2022, LEAD() retrieves 2023 sales.
- For 2023, it retrieves 2024 sales.
- For 2024, it retrieves 2025 sales.
- For 2025, no following row exists, so the result is NULL.

Use case: Comparing current sales with future-period sales.

Remember: LAG() looks backward; LEAD() looks forward.

4.3 FIRST_VALUE() — Get the First Value

Definition: Returns the first value in the window frame, based on the specified ordering.

Example

```
SELECT
  year,
```

```
  sales,
  FIRST_VALUE(sales) OVER (
    ORDER BY year
  ) AS FirstSales
FROM yearlySales;
```

Conceptual output:

Year	Sales	FirstSales
2022	10000	10000
2023	15000	10000
2024	12000	10000
2025	18000	10000

Explanation

The rows are ordered from the earliest year to the latest year. The first sales value is 10000, which belongs to 2022, so FIRST_VALUE() returns 10000 for each row in this example.

Use case: Comparing each year's sales with the sales value of the first year.

4.4 LAST_VALUE() — Get the Last Value

Definition: Returns the last value in the current window frame, based on the specified ordering.

Example

```
SELECT
  year,
  sales,
  LAST_VALUE(sales) OVER (
    ORDER BY year
    ROWS BETWEEN UNBOUNDED PRECEDING
        AND UNBOUNDED FOLLOWING
  ) AS LastSales
```

FROM yearlySales;

Conceptual output:

Year	Sales	LastSales
2022	10000	18000
2023	15000	18000
2024	12000	18000
2025	18000	18000

Explanation

- The rows are ordered by year.
- UNBOUNDED PRECEDING starts the frame at the first row.
- UNBOUNDED FOLLOWING extends the frame to the final row.
- Therefore, LAST_VALUE() returns the final sales value, 18000, for every row.

Important: The explicit frame is useful because the default frame for LAST_VALUE() with an ORDER BY generally ends at the current row or its peers. Without extending the frame to the final row, you might not get the last value of the entire ordered dataset.

Use case: Comparing every period's sales with the sales value of the final period.

5. Aggregation Functions

Aggregation functions calculate summary values. When used as window functions with OVER(), they preserve the individual rows in the result.

Your notes include SUM() and MIN().

5.1 SUM() — Calculate the Total

Definition: Calculates the sum of values across the specified window.

Example

```
SELECT
  customerName,
  country,
  creditLimit,
  SUM(creditLimit) OVER () AS TotalCreditLimit
```

```
FROM customers;
```

Explanation

- customerName displays the customer.
- country displays the customer's country.
- creditLimit displays the individual credit limit.
- SUM(creditLimit) OVER() calculates the total credit limit of all customers in the query result, then displays that total in each row.

You can also partition the calculation by country:

```
SELECT
  customerName,
  country,
  creditLimit,
  SUM(creditLimit) OVER (
    PARTITION BY country
  ) AS CountryCreditTotal
FROM customers;
```

Here, each customer keeps their own row, but the total is calculated separately for each country.

Understanding PARTITION BY: It divides the rows into separate windows. For example, all USA customers are in one window, all UK customers are in another, and all Indian customers are in another.

5.2 MIN() — Find the Minimum Value

Definition: Returns the smallest value within the specified window.

Example

```
SELECT
  customerName,
  country,
  creditLimit,
  MIN(creditLimit) OVER () AS MinimumCreditLimit
```

```
FROM customers;
```

Explanation

- creditLimit displays each customer's individual credit limit.
- MIN(creditLimit) OVER() finds the smallest credit limit among all customers in the result.
- The minimum value is repeated for each row.

You can calculate the minimum separately for each country:

```
SELECT
  customerName,
  country,
  creditLimit,
  MIN(creditLimit) OVER (
    PARTITION BY country
  ) AS CountryMinimumCredit
FROM customers;
```

This returns the lowest credit limit within each customer's country while preserving individual customer records.

6. Understanding OVER(), PARTITION BY, and ORDER BY

These clauses control how a window function works.

Clause	Purpose
OVER()	Defines the window used for the calculation.
PARTITION BY	Divides rows into separate windows or groups.
ORDER BY inside OVER()	Defines the order used for ranking or accessing rows.

Example A: Total across all customers

```
SUM(creditLimit) OVER ()
```

Calculates one overall total and repeats it on every row.

Example B: Total within each country

```
SUM(creditLimit) OVER (
```

```
  PARTITION BY country
)
```

Calculates a separate total for each country and repeats that country's total on its customer rows.

Example C: Rank customers by credit limit

```
RANK() OVER (
  ORDER BY creditLimit DESC
)
```

Ranks customers from the highest credit limit to the lowest.

7. Window Functions vs. GROUP BY

Both can calculate summaries, but they produce different result shapes.

Using GROUP BY:

```
SELECT
  country,
  SUM(creditLimit) AS CountryCreditTotal
FROM customers
GROUP BY country;
```

This produces one row per country.

Using a window function:

```
SELECT
  customerName,
  country,
  creditLimit,
  SUM(creditLimit) OVER (
    PARTITION BY country
  ) AS CountryCreditTotal
FROM customers;
```

This produces one row per customer, with that customer's country's total alongside it.

Feature	GROUP BY	Window function
Individual rows	Usually combined into groups	Preserved
Summary calculation	Yes	Yes
Can rank rows	Not directly through grouping alone	Yes, using ranking functions
Can compare with previous or next row	Not directly	Yes, using LAG() and LEAD()

8. Final Revision Table

Type	Function	Main purpose
Ranking	ROW_NUMBER()	Assigns a unique sequential row number
Ranking	RANK()	Ranks rows, leaving gaps after ties
Ranking	DENSE_RANK()	Ranks rows without gaps after ties
Ranking	NTILE()	Divides ordered rows into groups
Value	LAG()	Retrieves a previous row's value
Value	LEAD()	Retrieves a following row's value
Value	FIRST_VALUE()	Retrieves the first value in the window frame
Value	LAST_VALUE()	Retrieves the last value in the window frame
Aggregation	SUM()	Calculates a total
Aggregation	MIN()	Finds the minimum value

The key idea to remember: Window functions perform calculations or comparisons across rows while keeping the original individual rows visible. OVER() defines the calculation window, PARTITION BY separates that window into groups, and ORDER BY controls the row order when needed.